DSP IMP QUESTION

UNIT 1

Q1) List types of Data Structure / List out different Data Structure with examples (S-23, S-25)

A Data Structure is a systematic way to store, organize and manage data so that it can be used efficiently.
Each program uses different types of data structures depending on the need of searching, sorting or storing data.
Data structures are mainly classified into two major types:

1) Primitive Data Structures
These are basic data types provided by programming languages.

Examples:
Integer (int) → 10, 50, 100
Floating point (float) → 3.14, 6.28
Character (char) → 'A', 'B'
Boolean → True, False

These are the building blocks for non-primitive data structures.

2) Non-Primitive Data Structures
These store large and complex data in a structured form.

A) Linear Data Structures
Elements are arranged sequentially, one after another.

Examples:
1. Array
Collection of elements of same type
Example: a = [1,2,3,4]

2. List (Python List)
Can store mixed data
Example: [10, "hello", 3.5]

3. Stack
Follows LIFO (Last In First Out)
Example: Undo operation in editor

4. Queue
Follows FIFO (First In First Out)
Example: Ticket booking line

5. Linked List
Nodes connected using links
Example: Music playlist

B) Non-Linear Data Structures
Data is organized in hierarchical form.

Examples:
1. Tree
Parent-child relationship
Example: File directory structure

2. Graph
Nodes connected using edges
Example: Social networks

C) Hash-based Data Structure

1. Hash Table (Dictionary in Python)
Stores data in key:value format
Example:
student = {"id": 1, "name": "Ravi"}

Q2) Implement a program for two matrix multiplication using nested loop and NumPy (S-23, W-23)
Matrix multiplication is performed only when:
Number of columns in first matrix = Number of rows in second matrix.

Formula:
If
A = m × n
B = n × p
Result C = m × p
Each element of C is calculated as:
C[i][j] = A[i][0]*B[0][j] + A[i][1]*B[1][j] + ... + A[i][n-1]*B[n-1][j]

Program Using Nested Loops

```
# Matrix Multiplication using nested loop
A = [[1, 2],
     [3, 4]]

B = [[2, 0],
```

```
    [1, 2]]
# result matrix
C = [[0, 0],
    [0, 0]]
for i in range(len(A)):            # rows of A
   for j in range(len(B[0])):     # columns of B
       for k in range(len(B)):    # rows of B
           C[i][j] += A[i][k] * B[k][j]
print("Matrix Multiplication =", C)
```

Program Using NumPy

```
import numpy as np
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[2, 0],
              [1, 2]])
C = np.dot(A, B)
print("Result =", C)
```

Q3) Write a program to demonstrate array operations in Python (S-23, W-23)

Array is a collection of elements having the same data type.
Python provides array using the module array.

Common Operations
1. Creation
2. Insertion
3. Deletion
4. Traversal
5. Searching
6. Updating

Program

```
import array as arr
# create array of integers
a = arr.array('i', [10, 20, 30])
print("Original Array:", a)
# 1) Insert at end
a.append(40)
# 2) Insert at given position
a.insert(1, 15)  # insert 15 at index 1
```

```
# 3) Remove an element
a.remove(30)
# 4) Pop last element
a.pop()
# 5) Traversal
print("Array Elements:")
for i in a:
    print(i)
```

Q4) Explain Asymptotic Notations
(S-23)

Asymptotic notations describe the efficiency of an algorithm when the input size becomes very large.
They help to compare algorithms without depending on hardware or programming language.

1) Big O Notation – O(f(n))
Represents worst-case time.
Upper bound of growth rate.
Ensures that the running time never crosses this limit.

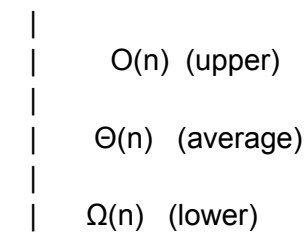
Example:
Linear Search → O(n)

2) Omega Notation – Ω(f(n))
Represents best-case time.
Lower bound of algorithm.

Example:
If first element is found in search → Ω(1)

3) Theta Notation – Θ(f(n))
Represents average-case.
Gives exact bound.
If an algorithm has same upper and lower bounds → Θ(n)

Graph Representation



|
----------> n (input size)

Importance
Helps to measure efficiency
Hardware independent
Helps in algorithm selection

Q5) Compare List and Tuple
(S-23)
Both List and Tuple are sequence data structures in Python, but they differ in many ways.

Definition
List: Mutable (modifiable) collection of items.
Tuple: Immutable (cannot be changed after creation).

Difference Table

Feature	List	Tuple
Mutability	Mutable (change allowed)	Immutable
Syntax	[ ]	( )
Performance	Slower	Faster
Memory Usage	More memory	Less memory
Use Case	Suitable for data that changes	Suitable for constant data
Methods	Many built-in methods	Very few methods
Example	[10, 20, 30]	(10, 20, 30)

Example
```
# List
l = [1, 2, 3]
l[1] = 10  # modification allowed
# Tuple
t = (1, 2, 3)
# t[1] = 10  # ERROR, tuple is immutable
```

Q6) Differentiate between Array and List
(S-24)
Array and List both store multiple values, but they differ in nature, performance and usage.

Definition
Array
A collection of elements of the same data type.

Provided using Python's array module.

List
A collection of elements of different data types.
Most flexible data structure in Python.

Difference Table

Feature	Array	List
Data Type	Stores same type data	Stores different type data
Size	Fixed-size (in other languages)	Dynamic (size increases automatically)
Speed	Faster for numerical operations	Slower compared to array
Memory	Uses less memory	Uses more memory
Import	Need to import array module	No import needed (built-in)
Syntax	array('i',[1,2,3])	[1,2,3]
Operations	Limited functions	Many built-in functions
Use	Mathematical operations	General-purpose data storage

Example

```
import array as arr
a = arr.array('i', [10, 20, 30])
l = [10, 20, "Hello", 4.5]
print(a)
print(l)
```

Q7) Explain Python-specific data structures: List, Tuple, Dictionary OR Explain Set Data Structure**
Python provides rich built-in data structures.

1) List
Ordered collection
Mutable (changeable)
Allows duplicates

Example:
l = [10, 20, 30]

Operations
Append
Insert
Remove
Sort

2) Tuple
Ordered collection
Immutable (cannot change)
Faster than list

Example:
t = (10, 20, 30)

3) Dictionary
Unordered collection of key : value pairs
Fast searching using key

Example:
d = {"id":1, "name":"Ravi"}

4) Set
Unordered collection
No duplicate values
Used for mathematical operations

Example:
s = {10, 20, 20, 30} # stores 10, 20, 30 only

Q8) Define Tuple in Python. Explain operations of Tuple data structure
Definition
Tuple is an ordered, immutable sequence of elements.
Once created, values cannot be changed.

Syntax:
t = (10, 20, 30)

Operations on Tuple

1) Accessing elements
t[0] # 10

2) Slicing
t[1:3]

3) Concatenation
t1 = (1,2)
t2 = (3,4)
t3 = t1 + t2

4) Repetition
t * 3

5) Membership
10 in t

6) Count & Index
t.count(10)
t.index(20)

7) Length
len(t)

Example Program

```
t = (10, 20, 30, 20)
print("Tuple:", t)
print("Length:", len(t))
print("Index of 30:", t.index(30))
print("Count of 20:", t.count(20))
```

Q9) Define Following terms: Time Complexity, Space Complexity, Best Case (S-24)

1) Time Complexity
Time complexity defines how much time an algorithm takes with respect to input size (n).
It is expressed using Big O notation.

Example:
Linear Search → O(n)

2) Space Complexity
Space complexity measures total memory used by an algorithm.
Includes:
Input space
Auxiliary space

Example:
Binary Search → O(1)

3) Best Case
Best case is the condition when an algorithm performs minimum work.

Example:

If the searched element is at 1st position:
Linear Search Best Case → O(1)

Q10) Differentiate between Linear and Non-Linear Data Structure (S-25)

Definition
Linear Data Structure
Data is arranged in sequence (one-by-one).

Non-Linear Data Structure
Data is arranged hierarchically, not in sequence.

Difference Table

Feature	Linear	Non-Linear
Structure	Sequential	Hierarchical
Traversal	Single level	Multi-level
Memory	Contiguous (usually)	Non-contiguous
Examples	Array, List, Stack, Queue	Tree, Graph
Implementation	Easy	Complex
Searching	Slow	Faster
Relationship	One-to-One	One-to-Many / Many-to-Many

Examples

Linear
[10, 20, 30] → list
10 → 20 → 30 → Linked list sequence

Non-Linear
A
/ \
B C

Q11) Define Best Case, Worst Case, Average Case OR Explain Big O Notation (S-25, W-23)

1) Best Case
Minimum time required by algorithm.
Example: Element found at first position.
Time → O(1)

2) Worst Case
Maximum time required by algorithm.

Example: Element found at last position or not found.
Time → O(n)

3) Average Case
Average time for all inputs.
Example: Expected position of search → middle.
Time → O(n/2) ≈ O(n)

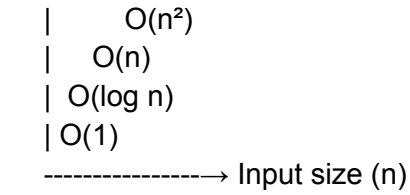
OR

Explain Big O Notation

Big O defines the upper bound of an algorithm.
It shows how the algorithm behaves when input size becomes very large.

Examples:
Constant time → O(1)
Linear → O(n)
Quadratic → O(n²)
Logarithmic → O(log n)

Graph



UNIT 2

Q1. Explain various types of methods in OOP. (S-23)

In Object-Oriented Programming (OOP), a class contains data (variables) and methods (functions). Methods define the behavior of an object. Python supports several types of methods based on how they access data and how they are used. The major types of methods in OOP are Instance Methods, Class Methods, Static Methods, Constructor Methods, and Special Methods.

1. Instance Method
Instance methods are the most commonly used methods in a class.
They work with instance variables and require an object of the class to be created.
They always have self as the first parameter, which refers to the current object.

Example:

```
class Student:
    def setData(self, name, age):
        self.name = name
        self.age = age
    def display(self):
        print(self.name, self.age)
```

2. Class Method
Class methods work with class variables and use the class name instead of objects.
They use @classmethod decorator and take cls as the first parameter.

Example:

```
class MyClass:
    count = 0
    @classmethod
    def showCount(cls):
        print("Count =", cls.count)
```

3. Static Method
Static methods do not work with class variables or instance variables.
They behave like normal functions but belong to the class.
They use @staticmethod decorator and do not use self or cls.

Example:

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b
```

4. Constructor Method
Constructor is a special method that initializes the object when it is created.
In Python, constructor is written using the __init__() method.

Example:

```
class Sample:
    def __init__(self):
        print("Object Created")
```

5. Special Methods (Magic/Dunder methods)

These methods begin and end with double underscores.
Examples: __str__(), __len__(), __add__() etc.

Q2. Explain types of inheritance. (S-23)

Inheritance allows one class to acquire the properties and behavior of another class.
The class that inherits is called Child Class, and the class from which it inherits is Parent Class.
Python supports the following types of inheritance:

1. Single Inheritance
A child class inherits from one parent class.

Example:

```
class A:
    pass
class B(A):
    pass
```

2. Multiple Inheritance
A child class inherits from more than one parent class.

```
class A:
    pass
class B:
    pass
class C(A, B):
    pass
```

3. Multilevel Inheritance
A child class becomes the parent of another class, forming a chain.

```
class A:
    pass
class B(A):
    pass
class C(B):
    pass
```

4. Hierarchical Inheritance

Multiple child classes inherit from a single parent class.

```
class A:
    pass
```

```
class B(A):
    pass
class C(A):
    pass
```

5. Hybrid Inheritance
Combination of more than one type of inheritance.

Example: A combination of single + multiple inheritance.

Q3. Design a Student class to read & display student info using getInfo() and displayInfo() methods. (S-21)

Below is a long descriptive answer with a proper class and methods.
In Python, we can create a Student class to store and display student details. The class contains two main methods:

1. getInfo() – to input the student details
2. displayInfo() – to print the student details

Program:

```
class Student:
    def getInfo(self):
        self.roll = int(input("Enter Roll No: "))
        self.name = input("Enter Name: ")
        self.course = input("Enter Course: ")
    def displayInfo(self):
        print("----- Student Information -----")
        print("Roll No:", self.roll)
        print("Name:", self.name)
        print("Course:", self.course)
# Main program
s = Student()
s.getInfo()
s.displayInfo()
```

This program demonstrates the use of instance methods and shows how OOP encapsulates data within a class.

Q4. Explain Polymorphism with example OR Write a program to demonstrate Polymorphism. (S-23, W-23, W-24)

Polymorphism means one task, many forms.
In OOP, polymorphism allows the same function name to behave differently depending on the object used.

Python supports two types of polymorphism:

1. Polymorphism with Method Overriding
Child class provides a new version of a method inherited from parent class.

```
class Animal:
    def sound(self):
        print("Animals make sound")
class Dog(Animal):
    def sound(self):
        print("Dog barks")
class Cat(Animal):
    def sound(self):
        print("Cat meows")
# Demonstration
for obj in [Animal(), Dog(), Cat()]:
    obj.sound()
```

2. Polymorphism with Functions
Different classes have same method names.

```
class Bird:
    def fly(self):
        print("Bird can fly")
class Airplane:
    def fly(self):
        print("Airplane flies high")
def show(obj):
    obj.fly()
show(Bird())
show(Airplane())
```

Q5. Write a Python program to demonstrate method overriding using inheritance. (S-23, W-23)

Method overriding occurs when a child class provides its own implementation of a method that is already defined in the parent class.

Program:

```
class Parent:
    def display(self):
        print("Display from Parent class")
class Child(Parent):
```

```
    def display(self):
        print("Display from Child class - Method Overridden")
# Main program
c = Child()
c.display()
```

Output will show the overridden child method.

Q6. Create a program to print factorial of given number using recursive function. (S-21)

Program:

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n-1)
num = int(input("Enter number: "))
print("Factorial =", factorial(num))
```

Q7. Explain concept of class and object with Python program. (S-24)

Class

A class is a blueprint/template for creating objects.

It contains variables (attributes) and methods (functions).

Example: a class Car that defines properties like color, model, and methods like start() or stop().

Object

An object is an instance of a class.

It represents a real-world entity and uses the class blueprint to store data.

Example Program:

```
class Car:
    def setData(self, model, color):
        self.model = model
        self.color = color
    def show(self):
        print("Model:", self.model)
        print("Color:", self.color)

# Object creation
c = Car()
c.setData("Honda City", "White")
c.show()
```

Q8. Define constructor. Discuss types of constructors with Python program. (S-24, W-23)

Definition of Constructor

A constructor is a special method in a class that gets executed automatically whenever an object of the class is created.

In Python, constructor is defined using the method __init__(self).

Its main purpose is to initialize object variables and allocate required resources.

Characteristics of Constructor

Name is always __init__().

Automatically invoked when object is created.

Used to initialize data members.

Cannot return values explicitly.

Always contains at least one parameter (self).

Types of Constructors in Python

1. Default Constructor

A default constructor does not accept any argument except self.

It initializes default values.

Example:

```
class Demo:
    def __init__(self):
        print("Default Constructor Called")
obj = Demo()
```

2. Parameterized Constructor

Accepts parameters to initialize object with user-defined values.

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def show(self):
        print("Name:", self.name)
        print("Age:", self.age)
s = Student("Rahul", 20)
s.show()
```

3. Destructor (Optional Concept)
Destructor (__del__()) is used to destroy object memory, but Python automatically handles memory management.

Q9. Define Polymorphism. Write Python program for polymorphism through inheritance. (S-24, S-25)

Definition
Polymorphism means one name, many forms.
A single function or method can behave differently depending on the object that is using it.

Python supports polymorphism using:
Method Overriding
Duck Typing
Operator Overloading

Polymorphism Through Inheritance
Achieved when the child class overrides the method of parent class.

Program:

```
class Shape:
    def area(self):
        print("Area of shape")
class Rectangle(Shape):
    def area(self):
        print("Area of Rectangle = length * breadth")
class Circle(Shape):
    def area(self):
        print("Area of Circle = π * r * r")
# Polymorphism demonstration
for obj in (Shape(), Rectangle(), Circle()):
    obj.area()
```

This shows that the same method name area() behaves differently for different objects.

Q10. Explain different concepts of OOP OR What is Class and Object in OOP? Give example. (S-25, W-23)

OOP (Object-Oriented Programming) is a programming methodology based on the concept of “objects”.
It helps make software more modular, reusable, and easy to maintain.

Major Concepts of OOP

1. Class

A class is a user-defined blueprint that contains variables and methods.
It does not occupy memory until objects are created.

Example:

```
class Car:
    def run(self):
        print("Car is running")
```

2. Object
An object is an instance of a class.
It stores real data and uses class methods.

```
c = Car()
c.run()
```

3. Encapsulation
Binding data and methods together inside a class.

4. Inheritance
One class acquires properties of another class.

5. Polymorphism
Same method behaves differently in different classes.

6. Abstraction
Hiding complex details and showing only essential features.

These concepts make programming easier, more efficient, and flexible.

Q11. Define Abstraction. Write Python program to understand abstract class. (S-25, W-24)

Definition of Abstraction
Abstraction is the process of hiding the internal implementation and showing only the necessary details.
It helps reduce complexity.

In Python, abstraction is implemented using Abstract Base Class (ABC) from the abc module.

Characteristics
Cannot create objects of abstract class.
Contains one or more abstract methods (methods without body).
Child class must override abstract methods.

Program:

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        print("Area of Square:", self.side * self.side)
s = Square(4)
s.area()
```

This program shows how abstract class enforces method overriding.

Q12. Differentiate between class method and static method. (W-23)

Class Method
Uses decorator @classmethod.
First argument is cls (class reference).
Can access and update class variables.
Called using class name or object.

```
class Demo:
    count = 0
    @classmethod
    def show(cls):
        print(cls.count)
```

Static Method
Uses decorator @staticmethod.
Does not use self or cls.
Acts like a normal function inside a class.
Cannot access class or instance variables directly.

```
class Demo:
    @staticmethod
    def add(a, b):
        return a + b
```

Difference Table

Class Method Static Method

Uses @classmethod	Uses @staticmethod
First parameter = cls	No required parameter
Access class variables	Cannot access class/instance variables
Logic related to class	Logic independent of class

Q13. Implement a class for single level inheritance using public and private type derivation. (W-23)

Explanation

Single level inheritance involves one parent class and one child class.

Python simulates private members using double underscore (__).

Program:

```
class Parent:
    def __init__(self):
        self.public_var = "Public Member"
        self.__private_var = "Private Member"
    def showParent(self):
        print(self.public_var)
        print("Accessing private in parent:", self.__private_var)
class Child(Parent):
    def showChild(self):
        print("Child accessing public:", self.public_var)
        # private member cannot be accessed directly
        # print(self.__private_var)   # Error
c = Child()
c.showParent()
c.showChild()
```

This demonstrates public and private members in inheritance.

Q14. Write Python program to implement multiple and hierarchical inheritance. (W-23)

Program:

```
# Multiple Inheritance
class A:
    def showA(self):
        print("Class A")
class B:
    def showB(self):
        print("Class B")
class C(A, B):
    def showC(self):
        print("Class C from A and B")
```

```
obj1 = C()
obj1.showA()
obj1.showB()
obj1.showC()
# Hierarchical Inheritance
class Parent:
    def display(self):
        print("Parent Class")
class Child1(Parent):
    pass
class Child2(Parent):
    pass
c1 = Child1()
c2 = Child2()
c1.display()
c2.display()
```

Q15. Explain Types of constructors in Python. Write Python program to multiply two numbers using static method. (W-24)

Types of Constructors

1. Default Constructor – No parameters.
2. Parameterized Constructor – Accepts parameters.

Program Using Static Method:

```
class MathOp:
    @staticmethod
    def multiply(a, b):
        return a * b

print("Multiplication =", MathOp.multiply(5, 6))
```

Static method suits this task because multiplication does not depend on class or object variables.

Q16. Define Data Encapsulation. List types of methods in Python. Write program for multilevel inheritance. (W-24)

Data Encapsulation

Encapsulation means binding data (variables) and methods (functions) together in a single unit called class.

It also protects data using access modifiers (public, private).

Advantages:

Data security

Data hiding
Avoids accidental modification
Improves modularity

Types of Methods in Python

1. Instance Methods
2. Class Methods
3. Static Methods
4. Constructor Method
5. Special (Magic) Methods

Program for Multilevel Inheritance:

```
class A:
    def showA(self):
        print("Class A")
class B(A):
    def showB(self):
        print("Class B derived from A")
class C(B):
    def showC(self):
        print("Class C derived from B")
obj = C()
obj.showA()
obj.showB()
obj.showC()
```

This demonstrates multilevel inheritance (A → B → C).

UNIT 3

Q1. Explain operations on Stack OR Explain applications of Stack (S-23, S-24, W-23)

Stack:
A stack is a linear data structure which follows the LIFO (Last In First Out) principle. This means the element inserted last is removed first.

Operations on Stack:

1. Push: Insert an element at the top of the stack.
2. Pop: Remove the element from the top of the stack.
3. Peek / Top: Access the top element without removing it.
4. IsEmpty: Check whether the stack is empty.
5. IsFull: Check whether the stack is full (in case of static stack).

Applications of Stack:
1. Expression evaluation (infix, postfix, prefix).
2. Function call management in recursion.
3. Undo mechanism in text editors.
4. Syntax parsing in compilers.
5. Browser back/forward navigation.

Q2. Explain Circular Queue OR Define circular queue and explain INSERT & DELETE operations with diagrams (S-23, S-24, S-25)

Circular Queue:
A circular queue is a linear data structure where the last position is connected to the first position, forming a circle. It overcomes the limitation of simple linear queues (wasted space after deletion).

Operations:

1. Insert (Enqueue):
If rear = last position and front ≠ 0 → set rear = 0.
Else rear = rear + 1 and insert element.

2. Delete (Dequeue):
Remove element from front.
If front = rear → queue becomes empty.
Else, if front = last position → front = 0.
Else → front = front + 1.

Diagram:

Circular Queue of size 5: [10, 20, 30, 40, 50]
front -> 10, rear -> 50
After Dequeue: front -> 20
After Enqueue(60): rear -> 0, queue = [60, 20, 30, 40, 50]

Q3. Summarize applications of Queue. Show limitations of Queue (S-23)

Queue:
A queue is a linear data structure following FIFO (First In First Out).

Applications:

1. CPU scheduling.
2. Printer job scheduling.
3. Handling requests in OS (process scheduling).
4. Customer service systems (bank, call center).
5. Data buffering (IO buffers, network buffers).

Limitations:
1. Fixed size in array implementation → cannot dynamically grow.
2. Wasted memory in simple linear queue after deletion.
3. Insertion is restricted to rear; deletion restricted to front (less flexible than deque).

Q4. Create a program to print factorial using recursion (S-21)
Python Program:

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n-1)
num = int(input("Enter a number: "))
print("Factorial of", num, "is", factorial(num))
```

Explanation:
Base case: factorial(0) = 1.
Recursive call: factorial(n) = n * factorial(n-1).

Q5. Differentiate circular and simple queues (S-23)

Feature	Simple Queue	Circular Queue
Structure	Linear	Circular (last connected to first)
Memory Utilization	Wastes space after deletion	Efficient, reuses space
Front & Rear	Front moves forward only	Front & Rear wrap around
Overflow	Occurs even if empty spaces exist	Only occurs when full
Example	[10, 20, 30, _, _] after deletion	[_, 20, 30, 40, 50]

Q6. Develop program to convert infix to postfix using Stack (S-23)
Algorithm:

1. Initialize an empty stack and postfix expression.
2. Scan infix expression from left to right.
3. If operand → add to postfix.
4. If '(' → push to stack.
5. If ')' → pop and add to postfix until '(' is found.
6. If operator → pop from stack while precedence(stack top) ≥ precedence(current) → add to postfix.
7. Push current operator.
8. After scanning, pop all remaining operators to postfix.

Python Code:

```
def precedence(op):
    if op in ('+', '-'): return 1
    if op in ('*', '/', '%'): return 2
    return 0
def infix_to_postfix(expression):
    stack = []
    postfix = ''
    for char in expression:
        if char.isalnum():  # operand
            postfix += char
        elif char == '(':
            stack.append(char)
        elif char == ')':
            while stack and stack[-1] != '(':
                postfix += stack.pop()
            stack.pop()
        else:  # operator
            while stack and precedence(stack[-1]) >= precedence(char):
                postfix += stack.pop()
            stack.append(char)
    while stack:
        postfix += stack.pop()
    return postfix

expr = "A+B*C"
print("Postfix:", infix_to_postfix(expr))
```

Q7. Define Stack. Explain PUSH & POP with example. Write algorithm for PUSH & POP (S-24, S-25, W-23, W-24)

Definition:
Stack is a LIFO data structure where insertion and deletion happen at the top.

PUSH Operation:

Adds element at the top of the stack.

Algorithm:

1. Check if stack is full → overflow.
2. Increment top.
3. stack[top] = new element.

POP Operation:

Removes element from the top of the stack.

Algorithm:

1. Check if stack is empty → underflow.
2. element = stack[top]
3. Decrement top
4. Return element

Example:

Stack = [10, 20, 30], top = 2
PUSH(40) → Stack = [10, 20, 30, 40], top = 3
POP() → returns 40, Stack = [10, 20, 30], top = 2

Q8. Convert Infix expression A = (B/C + (D)%E + F) / (G*H) into Postfix (S-24, S-25, W-23, W-24)

Step 1: Understand precedence

() → highest
*, /, % → highest
+, - → lower

Infix Expression:

A = (B / C + (D) % E + F) / (G * H)

Step 2: Convert using stack rules

1. Operand → add to postfix
2. (→ push to stack
3.) → pop until (

Step 3: Conversion Process

Infix Symbol	Stack	Postfix
B		B
/	/	B
C	/	BC
+	+	BC/

(	+ (	BC/
D	+ (	BC/D
%	+ (%	BC/D
E	+ (%	BC/DE
)	+	BC/DE%
+	+	BC/DE%+
F	+	BC/DE%+F
/	/	BC/DE%+F+
(	/ (	BC/DE%+F+
G	/ (	BC/DE%+F+G
*	/ (*	BC/DE%+F+G
H	/ (*	BC/DE%+F+GH
)	/	BC/DE%+F+GH*

Step 4: Final Postfix Expression:
BC/DE%+F+GH*/

Q9. Explain implementation of Stack using List (S-24)
Stack using List in Python:

Python lists provide dynamic array behavior → can act as a stack.

Push → append()
Pop → pop()

Python Code:

```
stack = []

# PUSH operation
stack.append(10)
stack.append(20)
stack.append(30)
print("Stack after PUSH:", stack)
# POP operation
element = stack.pop()
print("Popped element:", element)
print("Stack after POP:", stack)
```

Explanation:

append() adds element at end (top of stack).
pop() removes element from end (top of stack).

Advantages:
Dynamic size
Easy to implement

Disadvantages:
Slower if list is very large (due to resizing).

Q10. List applications of Queue (S-25)
Queue Applications:

1. CPU Scheduling → Ready Queue for processes.
2. Printer Spooling → Jobs are queued before printing.
3. Banking / Ticket Counter Systems → First come, first served.
4. Call Center Systems → Customer requests handled in order.
5. Data Buffers → Keyboard, IO buffers, and network buffers.
6. Breadth-First Search (BFS) in Graphs → Uses queue for traversal.

Q11. Explain Enqueue and Dequeue operations on Queue (W-23, W-24)
Queue Operations:

1. Enqueue (Insertion):
Adds an element at the rear of the queue.

Algorithm:

1. Check if queue is full → Overflow.
2. If empty → front = rear = 0
3. Else → rear = rear + 1
4. Add element at queue[rear]

2. Dequeue (Deletion):
Removes an element from the front of the queue.

Algorithm:

1. Check if queue is empty → Underflow.
2. Store queue[front] as deleted element
3. If front == rear → queue becomes empty → front = rear = -1
4. Else → front = front + 1
5. Return deleted element

Python Example:

```
queue = []

# Enqueue
queue.append(10)
queue.append(20)
queue.append(30)
# Dequeue
element = queue.pop(0)
print("Dequeued:", element)
print("Queue now:", queue)
```

Q12. Write algorithm for Dequeue operation using List (W-24)

Algorithm:

Algorithm Dequeue
1. Start
2. If queue is empty, print "Underflow" and stop
3. Else
   a. Store element = queue[front]
   b. If front == rear
      i. front = rear = -1 // Queue becomes empty
     Else
      i. front = front + 1
   c. Return element
4. End

Python Implementation:

```
queue = [10, 20, 30]
front = 0
rear = len(queue) - 1
if front > rear:
    print("Queue Underflow")
else:
    element = queue[front]
    front += 1
    print("Dequeued element:", element)
```

Q13. List out limitations of Single Queue (W-23)

Limitations of Simple Queue (Single Queue):

1. Wasted memory → After dequeue, the empty space cannot be reused.
2. Fixed size in array implementation → Cannot grow dynamically.
3. Insertion limited to rear and deletion limited to front → Less flexible.

4. Overflow occurs even when queue has empty spaces at the front.

Q14. Differentiate circular and simple queues (W-23, W-24)

Feature	Simple Queue	Circular Queue
Structure	Linear	Circular (last connects to first)
Memory utilization	Wastes space after deletion	Reuses empty space efficiently
Front & Rear movement	Front moves only forward	Front & Rear wrap around
Overflow	Happens even if spaces available	Happens only if queue full
Complexity	Simple to implement	Slightly complex

Q15. Write algorithm for Infix to Postfix expression and evaluate example (W-24)

Algorithm for Infix to Postfix:

1. Initialize empty stack and postfix expression
2. For each symbol in infix expression:
   a. If operand → add to postfix
   b. If '(' → push to stack
   c. If ')' → pop from stack to postfix until '(' is found
   d. If operator →
      While stack is not empty and precedence(stack top) >= precedence(current)
      pop and add to postfix
      Push current operator
3. After scanning, pop remaining operators from stack to postfix
4. End

Example:
Infix: (A+B)*C

Conversion Process:

Stack: empty, Postfix: empty

(→ push → Stack: (()

A → operand → Postfix: A

+ → push → Stack: ((, +)

B → operand → Postfix: AB

) → pop → Postfix: AB+ Stack: empty

* → push → Stack: *

C → operand → Postfix: AB+C

End → pop remaining → Postfix: AB+C*

Final Postfix:
AB+C*

UNIT 4

Q1. List types of Linked List (S-23, W-23)

Linked List: A linked list is a linear data structure where elements (nodes) are stored in memory and connected using pointers. Each node contains data and a pointer to the next node (or previous node in some types).

Types of Linked Lists:

1. Singly Linked List (SLL):
Each node has data and a pointer to the next node.
Traversal is only in one direction (forward).

2. Doubly Linked List (DLL):
Each node contains data, a pointer to the next node, and a pointer to the previous node.
Traversal is in both directions.

3. Circular Linked List (CLL):
Last node points back to the first node, forming a circular chain.
Can be singly circular or doubly circular.

4. Header Linked List / Linked List with a dummy node (less common):
First node (header) may not store data, used to simplify list operations.

Q2. Differentiate circular linked list and singly linked list (S-23, W-23)

Feature	Singly Linked List	Circular Linked List
Structure	Last node points to NULL	Last node points to first node
Traversal	Forward only	Forward only (loop continues infinitely)
Starting Point	Head node	Any node can be starting point
Memory	Slightly less	Slightly more due to circular logic
Useful for	Simple linear storage	Repeated traversal, round-robin tasks

Example 10 -> 20 -> 30 -> NULL 10 -> 20 -> 30 -> 10 (circular)

Q3. Write program to insert node at beginning & end of singly linked list (S-23, W-23)

Python Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class SinglyLinkedList:
    def __init__(self):
        self.head = None
    # Insert at beginning
    def insert_at_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
    # Insert at end
    def insert_at_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node
    # Display linked list
    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("NULL")
# Example Usage
sll = SinglyLinkedList()
sll.insert_at_beginning(10)
sll.insert_at_beginning(5)
sll.insert_at_end(20)
sll.display()
```

Output:

5 -> 10 -> 20 -> NULL

Q4. What is doubly linked list? Write short note / Explain with advantages (S-23, S-24, W-23, W-24)

Doubly Linked List (DLL):

Each node contains three parts:

1. Data → stores the element
2. Prev → pointer to previous node
3. Next → pointer to next node

Head node's prev is NULL and last node's next is NULL.

Advantages:

1. Traversal in both directions (forward and backward).
2. Easier insertion and deletion at both ends or in the middle.
3. Useful for implementing stacks, deques, and navigation systems.

Diagram:

NULL <- [Prev|Data|Next] <-> [Prev|Data|Next] <-> [Prev|Data|Next] -> NULL

Q5. Describe applications of Linked List (S-23, W-23)

Applications of Linked List:

1. Dynamic memory allocation → memory can grow/shrink during runtime.
2. Implementation of stacks and queues → using singly or doubly linked list.
3. Graph adjacency representation → adjacency lists are implemented using linked lists.
4. Polynomial representation → each term is stored as a node.
5. Browser history navigation → forward and backward traversal using DLL.
6. Undo/Redo feature in text editors → doubly linked list is ideal.
7. Music/video playlists → circular linked list for repeated playback.

Q6. Explain doubly linked list. Write algorithm to delete node from beginning (S-23)

Algorithm to delete first node in DLL:

Algorithm DeleteFirstDLL
1. Start
2. If head is NULL
    Print "List is empty"
    Stop
3. Else
    temp = head

```
    head = head.next
    If head != NULL
       head.prev = NULL
    Delete temp
4. End
```

Explanation:

Remove the first node and update head pointer.
Adjust prev pointer of the new head to NULL.
Delete memory of the removed node.

Python Implementation:

```
def delete_from_beginning(self):
   if self.head is None:
      print("List is empty")
      return
   temp = self.head
   self.head = self.head.next
   if self.head:
      self.head.prev = None
   del temp
```

Q7. Explain Circular Linked List with disadvantages / advantages (S-24, S-25, W-24)

Circular Linked List (CLL):

Last node points back to the first node.
Can be singly circular (only next) or doubly circular (both next and prev).
Useful when you want continuous traversal without worrying about NULL.

Advantages:

1. Efficient memory utilization → no NULL at end.
2. Any node can be used as starting point.
3. Suitable for round-robin scheduling.
4. Traversal can be repeated indefinitely.

Disadvantages:

1. More complex to implement than simple linked list.
2. Difficult to detect end of list → need extra conditions.
3. More pointer updates required → increased chance of errors.

Diagram:

[10] -> [20] -> [30] -> [10] (circular)

Q8. Write Python program to perform Insert in doubly linked list with diagram (S-24)

Python Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
class DoublyLinkedList:
    def __init__(self):
        self.head = None
    # Insert at end
    def insert_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node
        new_node.prev = temp
    # Display list
    def display(self):
        temp = self.head
        while temp:
            print(f"[{temp.prev.data if temp.prev else None} <- {temp.data} -> {temp.next.data if
temp.next else None}]", end=" ")
            temp = temp.next
        print()
# Example Usage
dll = DoublyLinkedList()
dll.insert_end(10)
dll.insert_end(20)
dll.insert_end(30)
dll.display()
```

Diagram:

None <- 10 -> 20 <- 20 -> 30 <- 30 -> None

Q9. Differentiate Singly and Doubly Linked List (S-24)

Feature	Singly Linked List (SLL)	Doubly Linked List (DLL)
Node Structure	Data + Next pointer	Data + Prev + Next pointer
Traversal	Forward only	Forward and backward
Memory Usage	Less	More (extra prev pointer)
Insertion/Deletion	Slower in middle	Faster, easier at both ends
Complexity	Simple	Slightly complex
Example	10 -> 20 -> 30 -> NULL	NULL <- 10 <-> 20 <-> 30 -> NULL

Q10. Write algorithm to insert new node at end of circular linked list (S-24)

Algorithm:

Algorithm InsertEndCLL
1. Start
2. Create new node with data
3. If head is NULL
    head = new node
    new_node.next = head
    Stop
4. Else
    temp = head
    while temp.next != head
        temp = temp.next
    temp.next = new_node
    new_node.next = head
5. End

Explanation:

Traverse until last node (node pointing to head).
Insert new node after last node and make it point to head.

Q11. Write Python program to delete first & last node from singly linked list (S-25)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class SinglyLinkedList:
    def __init__(self):
        self.head = None
    # Insert at end
```

```
    def insert_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node
    # Delete first node
    def delete_first(self):
        if self.head is None:
            print("List is empty")
            return
        temp = self.head
        self.head = self.head.next
        del temp
    # Delete last node
    def delete_last(self):
        if self.head is None:
            print("List is empty")
            return
        if self.head.next is None:
            del self.head
            self.head = None
            return
        temp = self.head
        while temp.next.next:
            temp = temp.next
        del temp.next
        temp.next = None
    # Display list
    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("NULL")
# Example Usage
sll = SinglyLinkedList()
sll.insert_end(10)
sll.insert_end(20)
sll.insert_end(30)
sll.display()
```

```
sll.delete_first()
sll.display()
sll.delete_last()
sll.display()
```

Output:

```
10 -> 20 -> 30 -> NULL
20 -> 30 -> NULL
20 -> NULL
```

Q12. Explain different operations on singly linked list (S-25)

Operations:

1. Traversal: Visit all nodes from head to NULL.

2. Insertion:
At beginning → update head pointer
At end → traverse to last node and add new node
At middle → traverse to position, adjust pointers

3. Deletion:
At beginning → update head
At end → traverse to second last, remove last node
At middle → adjust previous node pointer to skip deleted node

4. Search: Find a node containing specific data by traversal

5. Update: Modify data of a node after searching

Q13. Write algorithm to insert new node at end of doubly linked list (S-25)

Algorithm:

```
Algorithm InsertEndDLL
1. Start
2. Create new node with data
3. If head is NULL
    head = new node
    new_node.prev = None
    new_node.next = None
    Stop
4. Else
    temp = head
    while temp.next != NULL
```

temp = temp.next
temp.next = new_node
new_node.prev = temp
new_node.next = NULL
5. End

Q14. Write algorithm to traverse nodes in singly linked list (W-24)
Algorithm:

Algorithm TraverseSLL
1. Start
2. temp = head
3. While temp != NULL
   a. Print temp.data
   b. temp = temp.next
4. End

Explanation:
Start from head and move pointer to next until NULL is reached.
Print each node's data.

Q15. Explain applications of singly linked list (W-24)
Applications:

1. Dynamic memory allocation → Memory can grow/shrink as needed.
2. Implementation of stacks and queues → using nodes with next pointers.
3. Polynomial arithmetic → each term is a node in the list.
4. Hash chaining → Handling collisions in hash tables.
5. Graph adjacency lists → store neighbors of a vertex.
6. Undo/Redo in software → maintain operations history.
7. Music or video playlists → sequential access to songs/videos.

UNIT 5

Q1. Implement Bubble Sort algorithm / Explain bubble sort algorithm with example / Write an algorithm of Bubble Sort (S-23, S-25, W-24)
Bubble Sort:
Bubble sort is a simple comparison-based sorting algorithm.
It repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.
The process is repeated until the list is sorted.

Algorithm (Bubble Sort):

Algorithm BubbleSort
1. Start
2. Let A[0..n-1] be the array
3. For i = 0 to n-2
    For j = 0 to n-i-2
        If A[j] > A[j+1] then
            Swap A[j] and A[j+1]
4. End

Example: Sort A = [5, 2, 8, 3]

Trace Table:

Pass	Array Status
1	[2, 5, 3, 8]
2	[2, 3, 5, 8]
3	[2, 3, 5, 8]

Python Implementation:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n-1):
        for j in range(n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
arr = [5, 2, 8, 3]
bubble_sort(arr)
print("Sorted array:", arr)
```

Q2. Write an algorithm for Merge Sort / Implement Merge Sort / Explain merge sort with example (S-24, W-23, W-24)

Merge Sort:
Merge sort is a divide and conquer algorithm.
The array is divided into two halves, each half is sorted recursively, then merged into a single sorted array.

Algorithm (Merge Sort):

Algorithm MergeSort(A, low, high)
1. If low < high then
    a. mid = (low + high) / 2
    b. MergeSort(A, low, mid)

c. MergeSort(A, mid+1, high)
d. Merge(A, low, mid, high)
2. End

Algorithm Merge(A, low, mid, high)
1. Create temporary arrays L = A[low..mid], R = A[mid+1..high]
2. i = 0, j = 0, k = low
3. While i < len(L) and j < len(R):
If L[i] <= R[j]:
A[k] = L[i]; i += 1
Else:
A[k] = R[j]; j += 1
k += 1
4. Copy remaining elements of L, if any
5. Copy remaining elements of R, if any

Example:
Sort [38, 27, 43, 3, 9, 82, 10]

Divide: [38,27,43,3] [9,82,10]

Divide further: [38,27] [43,3] and [9,82] [10]

Merge step by step → [3,27,38,43,9,10,82] → [3,9,10,27,38,43,82]

Python Implementation:

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr)//2
        L = arr[:mid]
        R = arr[mid:]
        merge_sort(L)
        merge_sort(R)
        i = j = k = 0
        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                arr[k] = L[i]; i += 1
            else:
                arr[k] = R[j]; j += 1
            k += 1
        while i < len(L):
            arr[k] = L[i]; i += 1; k += 1
        while j < len(R):
```

```
            arr[k] = R[j]; j += 1; k += 1
arr = [38, 27, 43, 3, 9, 82, 10]
merge_sort(arr)
print("Sorted array:", arr)
```

Q3. Write an algorithm for Selection Sort / Give trace for data: 13, 2, 6, 54, 18, 42, 11 (S-24)

Selection Sort:
Finds the minimum (or maximum) element in unsorted array and places it at the correct position.

Algorithm (Selection Sort):

Algorithm SelectionSort
1. Start
2. For i = 0 to n-2:
    min_index = i
    For j = i+1 to n-1:
        If A[j] < A[min_index]:
            min_index = j
    Swap A[i] and A[min_index]
3. End

Example Array: [13, 2, 6, 54, 18, 42, 11]

Trace Table:

Pass	Array Status
1	[2, 13, 6, 54, 18, 42, 11]
2	[2, 6, 13, 54, 18, 42, 11]
3	[2, 6, 11, 54, 18, 42, 13]
4	[2, 6, 11, 13, 18, 42, 54]
5	[2, 6, 11, 13, 18, 42, 54]
6	[2, 6, 11, 13, 18, 42, 54]

Q4. Write an algorithm for bubble sort / Trace for: 37, 22, 64, 84, 58, 52, 11 (S-24)

Bubble Sort Algorithm: (same as Q1)

Trace Table:

Pass	Array Status
1	[22, 37, 64, 58, 52, 11, 84]
2	[22, 37, 58, 52, 11, 64, 84]
3	[22, 37, 52, 11, 58, 64, 84]

4 [22, 37, 11, 52, 58, 64, 84]
5 [22, 11, 37, 52, 58, 64, 84]
6 [11, 22, 37, 52, 58, 64, 84]

Explanation:

In each pass, the largest unsorted element “bubbles” to its correct position.

Q5. Explain Binary Search Tree and its applications (S-25)

Binary Search Tree (BST):

A binary tree where for each node:
Left child < Parent
Right child > Parent

Enables efficient searching, insertion, deletion in O(log n) time for balanced BSTs.

Example:

```
50
      /  \
    30    70
   / \    / \
  20 40  60 80
```

Applications of BST:

1. Searching data efficiently
2. Sorting data
3. Implementing dynamic sets
4. Maintaining ordered data (e.g., dictionary)
5. Databases & file systems indexing

Q6. Write Python Program for Linear Search and explain with example

Linear Search:
Sequentially checks each element until the target is found.

Python Program:

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

```
arr = [10, 25, 30, 45, 50]
target = 30
result = linear_search(arr, target)
if result != -1:
    print(f"Element {target} found at index {result}")
else:
    print(f"Element {target} not found")
```

Explanation:

Checks each element from start to end.

Returns the index of the element if found, else -1.

Q7. Differentiate between Linear Search and Binary Search (S-24)

Feature	Linear Search	Binary Search
Technique	Sequential	Divide and conquer
Requirement	No sorting needed	Array must be sorted
Time Complexity	O(n)	O(log n)
Best Case	O(1)	O(1)
Worst Case	O(n)	O(log n)
Implementation	Simple	Slightly complex
Example	Search 30 in [10,25,30,45]	Search 30 in [10,20,30,40,50]

Q8. Explain Quick Sort algorithm with example (S-25)

Quick Sort:

Quick Sort is a divide and conquer sorting algorithm.

Steps:

1. Select a pivot element from the array.
2. Partition the array into two sub-arrays:
Left → elements < pivot
Right → elements > pivot
3. Recursively apply Quick Sort to the sub-arrays.

Algorithm (Quick Sort):

Algorithm QuickSort(A, low, high)
1. If low < high then
    a. pi = Partition(A, low, high)

```
    b. QuickSort(A, low, pi-1)
    c. QuickSort(A, pi+1, high)
2. End

Algorithm Partition(A, low, high)
1. pivot = A[high]
2. i = low - 1
3. For j = low to high-1
    If A[j] <= pivot
        i = i + 1
        Swap A[i] and A[j]
4. Swap A[i+1] and A[high]
5. Return i+1
```

Example: Sort [10, 7, 8, 9, 1, 5]
Pivot = 5 → Partition → [1, 5, 7, 8, 9, 10]
Recursively sort left [1] and right [7,8,9,10]
Final sorted array: [1,5,7,8,9,10]

Python Implementation:

```
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i+1], arr[high] = arr[high], arr[i+1]
    return i+1
def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi-1)
        quick_sort(arr, pi+1, high)
arr = [10, 7, 8, 9, 1, 5]
quick_sort(arr, 0, len(arr)-1)
print("Sorted array:", arr)
```

Q9. Write Python program for Insertion Sort with example (S-25)

Insertion Sort:
Builds the sorted array one element at a time.
Inserts each element into its correct position in the sorted part of the array.

Algorithm (Insertion Sort):

```
Algorithm InsertionSort(A)
1. For i = 1 to n-1
     key = A[i]
     j = i - 1
     While j >= 0 and A[j] > key
        A[j+1] = A[j]
        j = j - 1
     A[j+1] = key
2. End
```

Python Program:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
arr = [25, 15, 30, 9, 99, 20, 26]
insertion_sort(arr)
print("Sorted array:", arr)
```

Output:
[9, 15, 20, 25, 26, 30, 99]

Q10. Write an algorithm for Binary Search using list (W-24)

Binary Search:
Efficient search algorithm for sorted arrays.
Repeatedly divides the search space in half until the target element is found or the array is exhausted.

Algorithm (Binary Search):

```
Algorithm BinarySearch(A, target)
1. low = 0, high = n-1
2. While low <= high
     mid = (low + high) // 2
     If A[mid] == target
        Return mid
```

```
    Else if A[mid] < target
        low = mid + 1
    Else
        high = mid - 1
3. Return -1 (element not found)
```

Python Implementation:

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
arr = [10, 20, 30, 40, 50]
target = 30
result = binary_search(arr, target)
if result != -1:
    print(f"Element {target} found at index {result}")
else:
    print(f"Element {target} not found")
```

Q11. Define Search? Write an algorithm for Linear Search using list (W-24)

Definition of Search:
Search is the process of finding the location or presence of a specific element in a data structure.

Linear Search:
Sequentially checks each element until the target is found.

Algorithm (Linear Search):

```
Algorithm LinearSearch(A, target)
1. For i = 0 to n-1
    If A[i] == target
        Return i
2. Return -1 (element not found)
```

Python Implementation:

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
arr = [5, 10, 15, 20, 25]
target = 20
result = linear_search(arr, target)
if result != -1:
    print(f"Element {target} found at index {result}")
else:
    print(f"Element {target} not found")
```

UNIT 6

Q1. Describe applications of binary tree (S-23, W-23, W-24)

Binary Tree:
A binary tree is a hierarchical data structure where each node has at most two children, commonly referred to as the left child and right child.

Applications of Binary Tree:

1. Expression Trees – Used in compilers to represent arithmetic expressions; leaf nodes are operands and internal nodes are operators.
2. Binary Search Trees (BSTs) – Efficient searching, insertion, and deletion operations for sorted data.
3. Priority Queues & Heaps – Binary heap is used to implement priority queues.
4. Huffman Encoding Trees – Used in data compression algorithms.
5. Decision Trees – Used in AI and machine learning for decision-making.
6. Routing Tables – Used in network routers for efficient searching of addresses.
7. File System Hierarchies – Represent folder and file structures.

Q2. Explain Indegree and Outdegree of Binary Tree with example / Define level, degree and leaf node in binary tree (S-23, W-23)

1. Indegree & Outdegree:
Indegree: Number of edges coming into a node.
Outdegree: Number of edges going out from a node.

Example:

```
 / \
5   20
```

Node 10 → Indegree = 0, Outdegree = 2
Node 5 → Indegree = 1, Outdegree = 0 (leaf)
Node 20 → Indegree = 1, Outdegree = 0 (leaf)

2. Level of a Node:
Level of a node = Number of edges from root node to that node.
Root node level = 0

3. Degree of a Node:
Number of children a node has.
Example: Node 10 has degree 2, Node 5 has degree 0

4. Leaf Node:
A node with no children.
Example: 5 and 20 are leaf nodes in the above tree

Q3. Write a program to implement construction of binary search trees (S-21)
Python Program:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None
    # Insert a node
    def insert(self, root, key):
        if root is None:
            return Node(key)
        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        return root
    # Inorder Traversal
    def inorder(self, root):
        if root:
            self.inorder(root.left)
```

```
            print(root.key, end=" ")
            self.inorder(root.right)
# Example usage
bst = BST()
root = None
keys = [50, 30, 70, 20, 40, 60, 80]
for key in keys:
    root = bst.insert(root, key)
print("Inorder Traversal of BST:")
bst.inorder(root)
```

Output:
20 30 40 50 60 70 80

Explanation:
Elements are inserted to maintain the BST property: left child < parent < right child.
Inorder traversal gives sorted order.

Q4. Explain level and leaf node in binary tree (S-23, W-23)
Level of a Node:
The distance from the root node in terms of edges.
Root node is at level 0.

Example:

```
10 (Level 0)
     /  \
    5   20 (Level 1)
   /
  2  (Level 2)
```

Leaf Node:

A node that has no children.
In above tree, leaf nodes = 2 and 20

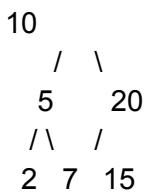
Applications of Leaf Nodes:

Represent end data in decision trees, expression trees, file structures.

Q5. Explain complete binary tree with example (S-23, W-23)
Complete Binary Tree (CBT):

A binary tree in which all levels are completely filled except possibly the last level, which is filled from left to right.

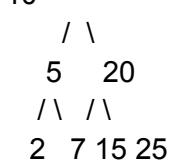
Example:



All levels except last are completely filled
Last level nodes (2,7,15) are filled from left to right

Applications:
Efficient representation of heaps
Used in priority queues

Q6. Perform Inorder, Preorder, Postorder traversal of given binary tree (S-23)
Given Binary Tree:



Traversals:

1. Inorder (Left, Root, Right):
2 5 7 10 15 20 25

2. Preorder (Root, Left, Right):
10 5 2 7 20 15 25

3. Postorder (Left, Right, Root):
2 7 5 15 25 20 10

Python Example for Traversals:

```
def inorder(root):
    if root:
        inorder(root.left)
        print(root.key, end=" ")
```

```
        inorder(root.right)

def preorder(root):
    if root:
        print(root.key, end=" ")
        preorder(root.left)
        preorder(root.right)

def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.key, end=" ")
```

Q7. Explain Binary search tree and application of it (S-24)

Binary Search Tree (BST):

A binary tree where:
Left child < Parent
Right child > Parent

Properties:
Efficient search, insertion, deletion operations
Inorder traversal gives sorted elements

Applications:

1. Searching – Find elements quickly in O(log n) time (average).
2. Sorting – Inorder traversal produces sorted order.
3. Database indexing – To quickly retrieve records.
4. Implementation of dictionaries and sets – BST maintains order.
5. Huffman coding tree – For data compression

Example BST:

```
50
      /  \
    30    70
   / \    / \
 20  40  60  80
```

Q8. Explain Quick Sort algorithm with example (S-25)

Quick Sort:
Quick Sort is a divide and conquer sorting algorithm.

Steps:

1. Choose a pivot element from the array.
2. Partition the array into two parts:
Elements less than pivot → left
Elements greater than pivot → right
3. Recursively apply Quick Sort to the left and right subarrays.

Algorithm (Quick Sort):

Algorithm QuickSort(A, low, high)
1. If low < high then
   a. pi = Partition(A, low, high)
   b. QuickSort(A, low, pi-1)
   c. QuickSort(A, pi+1, high)
2. End

Algorithm Partition(A, low, high)
1. pivot = A[high]
2. i = low - 1
3. For j = low to high-1
   If A[j] <= pivot
      i = i + 1
      Swap A[i] and A[j]
4. Swap A[i+1] and A[high]
5. Return i+1

Example: Sort [10, 7, 8, 9, 1, 5]

Pivot = 5 → Partition → [1, 5, 7, 8, 9, 10]
Recursively sort left [1] and right [7,8,9,10]
Sorted array: [1,5,7,8,9,10]

Python Implementation:

```
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i+1], arr[high] = arr[high], arr[i+1]
```

```
    return i+1

def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi-1)
        quick_sort(arr, pi+1, high)

arr = [10, 7, 8, 9, 1, 5]
quick_sort(arr, 0, len(arr)-1)
print("Sorted array:", arr)
```

Q9. Write Python program for Insertion Sort with example (S-25)

Insertion Sort:
Sorts an array by inserting each element at its correct position in the sorted portion.

Algorithm:

```
Algorithm InsertionSort(A)
1. For i = 1 to n-1
    key = A[i]
    j = i - 1
    While j >= 0 and A[j] > key
        A[j+1] = A[j]
        j = j - 1
    A[j+1] = key
2. End
```

Python Program:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key

arr = [25, 15, 30, 9, 99, 20, 26]
insertion_sort(arr)
print("Sorted array:", arr)
```

Output:

[9, 15, 20, 25, 26, 30, 99]

Q10. Write an algorithm for Binary Search using list (W-24)

Binary Search:
Efficient algorithm for sorted arrays.
Repeatedly divides the search interval in half.

Algorithm:

Algorithm BinarySearch(A, target)
1. low = 0, high = n-1
2. While low <= high
   mid = (low + high) // 2
   If A[mid] == target
      Return mid
   Else if A[mid] < target
      low = mid + 1
   Else
      high = mid - 1
3. Return -1

Python Program:

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

arr = [10, 20, 30, 40, 50]
target = 30
result = binary_search(arr, target)
print("Element found at index:", result if result != -1 else "Not Found")
```

Q11. Define Search? Write an algorithm for Linear Search using list (W-24)

Search:
Process of finding the location or presence of a specific element in a data structure.

Linear Search:
Checks each element sequentially until target is found.

Algorithm:

Algorithm LinearSearch(A, target)
1. For i = 0 to n-1
   If A[i] == target
      Return i
2. Return -1

Python Program:

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

arr = [5, 10, 15, 20, 25]
target = 20
result = linear_search(arr, target)
print("Element found at index:", result if result != -1 else "Not Found")
```

Q12. Explain Merge Sort algorithm with example (S-24, W-23, W-24)
Merge Sort:
Divide and conquer algorithm.
Divides array into halves, sorts each half, merges sorted halves.

Algorithm:

Algorithm MergeSort(A, low, high)
1. If low < high
   mid = (low + high) // 2
   MergeSort(A, low, mid)
   MergeSort(A, mid+1, high)
   Merge(A, low, mid, high)

Algorithm Merge(A, low, mid, high)
1. L = A[low..mid], R = A[mid+1..high]
2. i = j = 0, k = low
3. While i < len(L) and j < len(R)
   If L[i] <= R[j]

```
        A[k] = L[i]; i += 1
    Else
        A[k] = R[j]; j += 1
    k += 1
```
4. Copy remaining elements of L and R

Python Program:

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr)//2
        L = arr[:mid]
        R = arr[mid:]
        merge_sort(L)
        merge_sort(R)
        i = j = k = 0
        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                arr[k] = L[i]; i += 1
            else:
                arr[k] = R[j]; j += 1
            k += 1
        while i < len(L):
            arr[k] = L[i]; i += 1; k += 1
        while j < len(R):
            arr[k] = R[j]; j += 1; k += 1

arr = [38, 27, 43, 3, 9, 82, 10]
merge_sort(arr)
print("Sorted array:", arr)
```

Q13. Write an algorithm for Selection Sort and trace for given data (13, 2, 6, 54, 18, 42, 11) (S-24)

Selection Sort:
Finds the minimum element in the unsorted array and swaps it with the first unsorted element.

Algorithm:

```
Algorithm SelectionSort(A)
1. For i = 0 to n-2
    min_index = i
    For j = i+1 to n-1
        If A[j] < A[min_index]
            min_index = j
```

Swap A[i] and A[min_index]
2. End

Trace Table:

Pass	Array Status
1	[2, 13, 6, 54, 18, 42, 11]
2	[2, 6, 13, 54, 18, 42, 11]
3	[2, 6, 11, 54, 18, 42, 13]
4	[2, 6, 11, 13, 18, 42, 54]
5	[2, 6, 11, 13, 18, 42, 54]
6	[2, 6, 11, 13, 18, 42, 54]

Q14. Write an algorithm for Bubble Sort and trace for data (37, 22, 64, 84, 58, 52, 11) (S-24)
Bubble Sort Algorithm: (see Q1)

Trace Table:

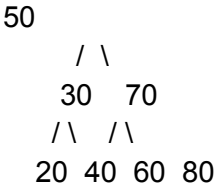
Pass	Array Status
1	[22, 37, 64, 58, 52, 11, 84]
2	[22, 37, 58, 52, 11, 64, 84]
3	[22, 37, 52, 11, 58, 64, 84]
4	[22, 37, 11, 52, 58, 64, 84]
5	[22, 11, 37, 52, 58, 64, 84]
6	[11, 22, 37, 52, 58, 64, 84]

Q15. Explain Binary Search Tree and applications (S-24)
Binary Search Tree (BST):
Binary tree where left child < parent < right child.

Properties:
Supports efficient search, insertion, deletion.
Inorder traversal gives sorted order.

Applications:
1. Searching elements quickly (O(log n) for balanced BST)
2. Sorting data
3. Database indexing
4. Implementation of dictionaries and sets
5. Decision-making algorithms

Example BST:

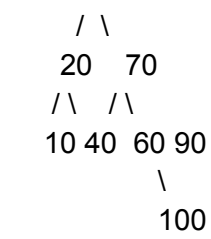


Q16. Construct a Binary Search Tree (BST) for sequence: 50, 70, 60, 20, 90, 10, 40, 100

Step 1: Insert keys maintaining BST property

Sequence: 50, 70, 60, 20, 90, 10, 40, 100

BST Structure:



Step 2: Traversals

1. Inorder (Left, Root, Right):

10 20 40 50 60 70 90 100

2. Preorder (Root, Left, Right):

50 20 10 40 70 60 90 100

3. Postorder (Left, Right, Root):

10 40 20 60 100 90 70 50

Python Implementation:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

```
def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    return root

def inorder(root):
    if root:
        inorder(root.left)
        print(root.key, end=" ")
        inorder(root.right)

keys = [50, 70, 60, 20, 90, 10, 40, 100]
root = None
for key in keys:
    root = insert(root, key)

print("Inorder traversal:")
inorder(root)
```

Q17. Define Tree. Enlist Types of Tree. Write algorithm to insert node in BST using Python

Definition:
A tree is a hierarchical data structure with a root node and zero or more child nodes, where each child has exactly one parent.

Types of Trees:

1. General Tree
2. Binary Tree
3. Binary Search Tree (BST)
4. AVL Tree (Balanced BST)
5. Heap (Min/Max)
6. Expression Tree
7. Decision Tree

Python Algorithm to Insert Node in BST:

```
def insert(root, key):
    if root is None:
```

```
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    return root
```

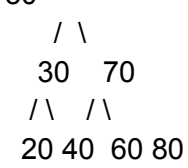
Q18. Write an algorithm for in-order traversal of tree

Inorder Traversal: Left → Root → Right

Algorithm:

Algorithm InorderTraversal(node)
1. If node == NULL
   return
2. InorderTraversal(node.left)
3. Visit node (print node.key)
4. InorderTraversal(node.right)

Example:



Inorder Traversal: 20 30 40 50 60 70 80

Python Implementation:

```
def inorder(root):
    if root:
        inorder(root.left)
        print(root.key, end=" ")
        inorder(root.right)
```

Q19. Define: (a) Path, (b) Leaf Node. Construct BST for → 60, 40, 37, 31, 59, 21, 65, 30

(a) Path:

A sequence of nodes connecting the root node to a particular node.

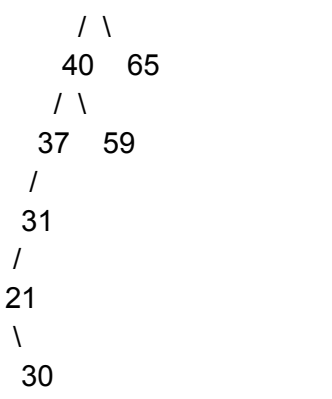
(b) Leaf Node:

Node with no children (degree = 0).

Step 1: Construct BST

Sequence: 60, 40, 37, 31, 59, 21, 65, 30

BST Structure:



Leaf Nodes: 30, 59, 65

Step 2: Traversals

1. Inorder (Left, Root, Right):

21 30 31 37 40 59 60 65

2. Preorder (Root, Left, Right):

60 40 37 31 21 30 59 65

3. Postorder (Left, Right, Root):

30 21 31 59 37 65 40 60

Python Implementation:

```
keys = [60, 40, 37, 31, 59, 21, 65, 30]
root = None
for key in keys:
    root = insert(root, key)

print("Inorder traversal:")
```

inorder(root)